How Transformers Work

Leah Lu

2025-07-02

Outline

- **"Attention Is All You Need" (NIPS 2017 paper)**
  - Transformers in context
  - Transformer architecture
  - Applications: machine translation, BERT, GPT
- **Deep Dive on Embeddings & Attention Mechanics**
  - Token and embeddings
  - Key, query, value roles in attention
- **Transformer Architecture and GenAI models**
  - Architecture for LLM models
  - Mixture of Experts structure and efficiency gains
  - From transformer to GPT assistant pipeline
- **Take-home Messages**
- **Q&A**

Attention Is All You Need

Ashish Vaswani* Google Brain avaswani@google.com

Noam Shazeer* Google Brain noam@google.com

Niki Parmar* Google Research nikip@google.com

Jakob Uszkoreit* Google Research usz@google.com

Llion Jones* Google Research llion@google.com

Aidan N. Gomez* † University of Toronto aidan@cs.toronto.edu

Łukasz Kaiser* Google Brain lukaszkaiser@google.com

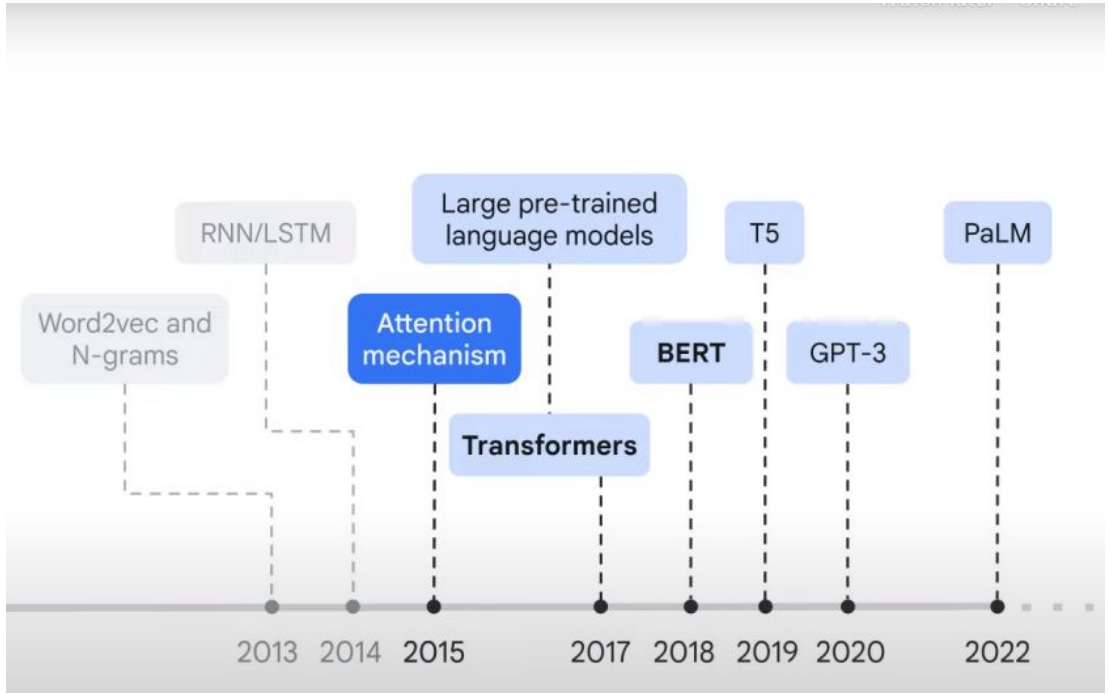
Illia Polosukhin* ‡ illia.polosukhin@gmail.com

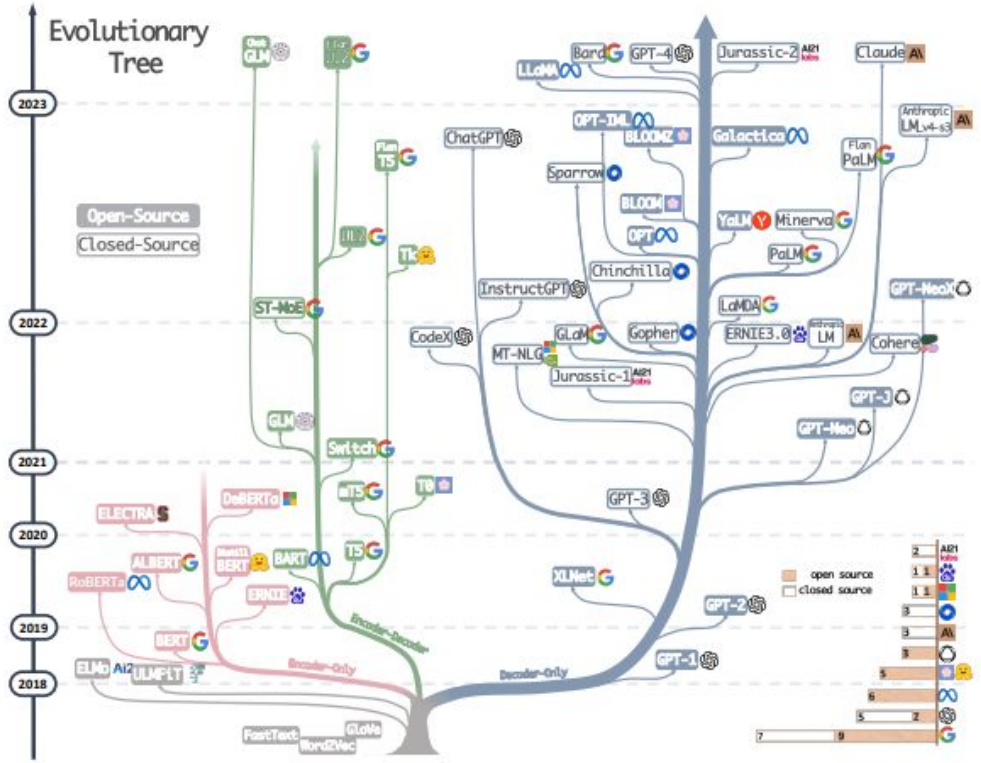
Abstract

The dominant sequence transduction models are based on complex recurrent or convolutional neural networks that include an encoder and a decoder. The best performing models also connect the encoder and decoder through an attention mechanism. We propose a new simple network architecture, the Transformer, based solely on attention mechanisms, dispensing with recurrence and convolutions entirely. Experiments on two machine translation tasks show these models to be superior in quality while being more parallelizable and requiring significantly less time to train. Our model achieves 28.4 BLEU on the WMT 2014 English-to-German translation task, improving over the existing best results, including ensembles, by over 2 BLEU. On the WMT 2014 English-to-French translation task, our model establishes a new single-model state-of-the-art BLEU score of 41.8 after training for 3.5 days on eight GPUs, a small fraction of the training costs of the best models from the literature. We show that the Transformer generalizes well to other tasks by applying it successfully to English constituency parsing both with large and limited training data.

31st Conference on Neural Information Processing Systems (NIPS 2017), Long Beach, CA, USA.

Transformers in Context





Transformers

What is a Language Model?

- To predict the probability for the next word, based on the previous ones
- Learns the probability distribution of word sequences, enabling tasks like text generation, translation, and autocomplete

$$P(x_1, x_2, ..., x_T) = \prod_{t=1}^{T} P(x_t \mid x_1, x_2, ..., x_{t-1})$$

“I grew up in France … I speak fluent” □ *French*

Self-Supervised Learning

- a model learns from unlabeled data by creating its own training signals (pseudo-labels) from the data itself
- training strategy used to build language models

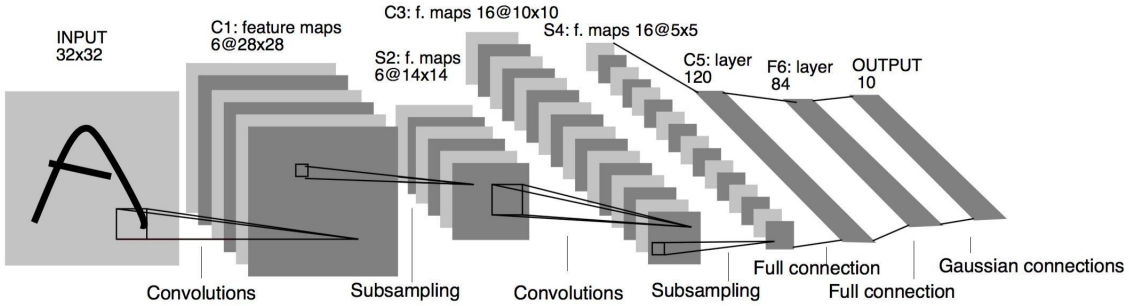
Masked Language Modeling (BERT-style)

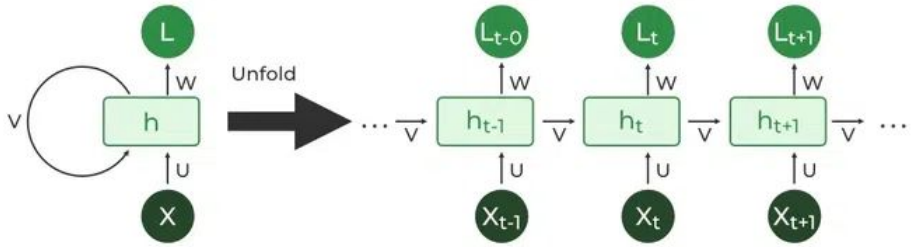
"The **MASK** didn't cross the road because it was tired." □ "animal"

Next Token Prediction (GPT-style)

"I grew up in France and I speak fluent" □ "French"

Before Transformer (2017)

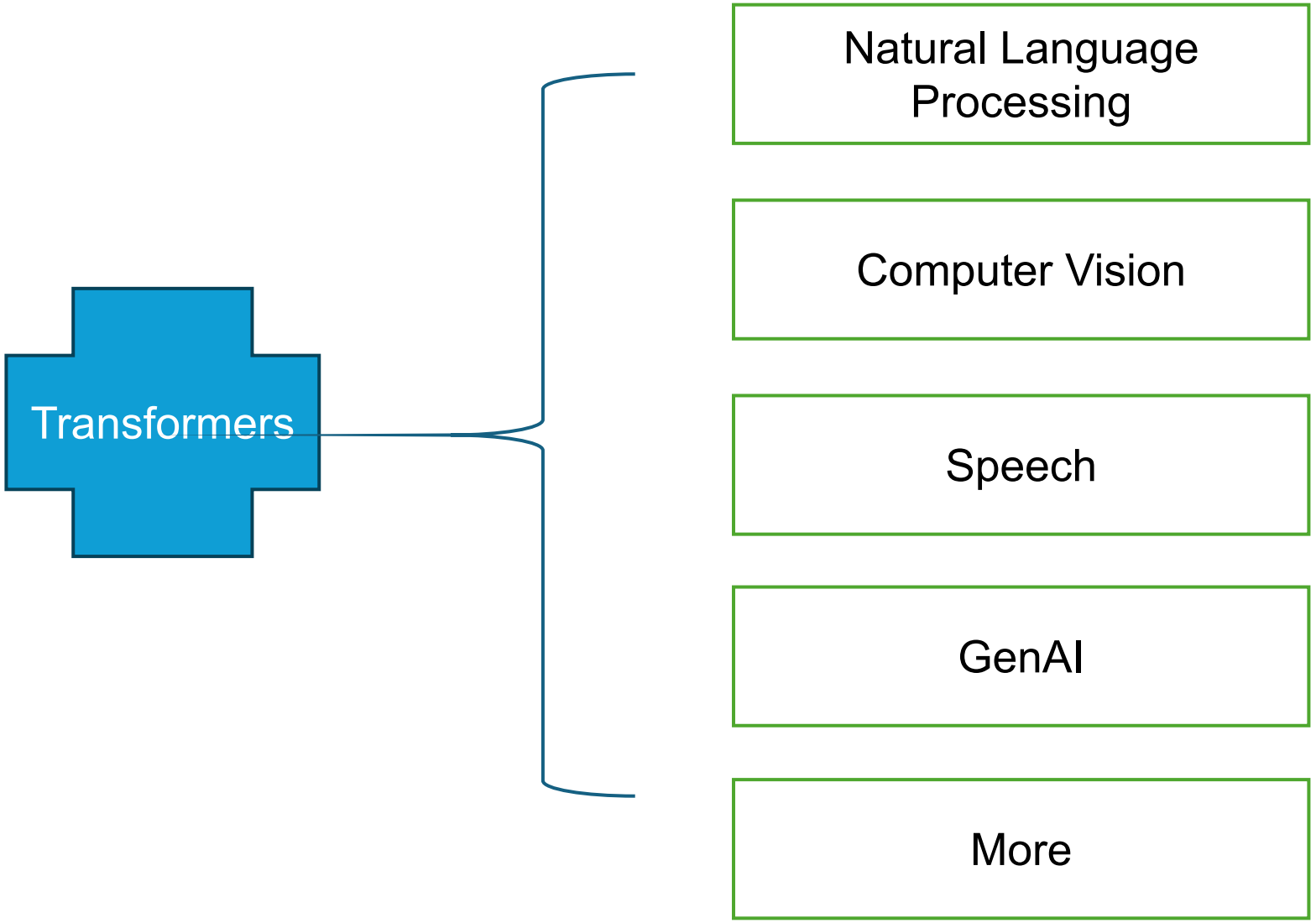




- **Computer Vision with Convolutional Neural Networks (CNNs)**
  - Input: Images
  - Output: Classification, object detection, segmentation
  - Architecture components: Convolution layers, Pooling / Subsampling, Fully connected layers
  - CNN Variants: AlexNet, VGG, ResNet

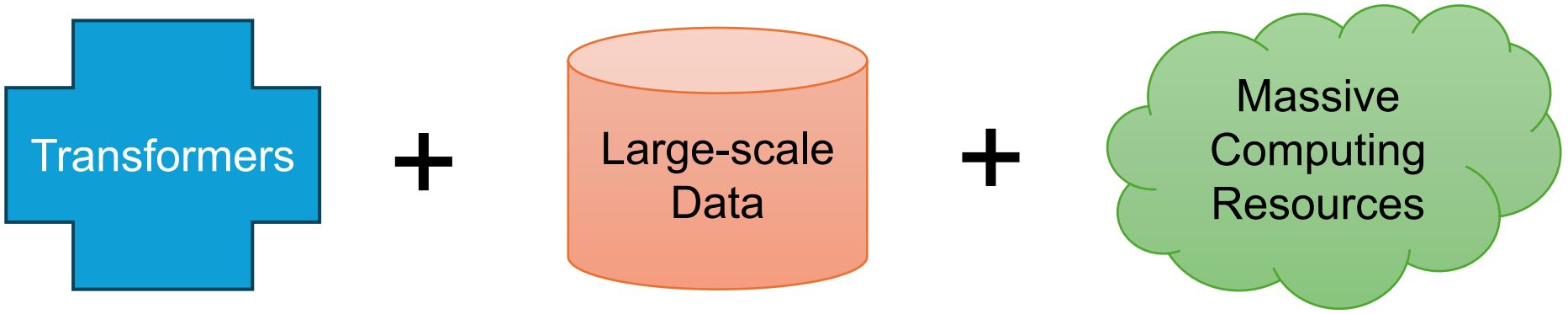
- **Natural Language with Recurrent Neural Networks (RNNs)**
  - Input: Text sequences
  - Output: Classification, translation, language modeling
  - Architecture components: recurrent connections, Sequential processing of tokens
  - RNN Variants: LSTM, GRU

After Transformer



Why Transformers?

- Attention → long-range dependencies
- Parallelization → efficiency
- Scalability → handle large scale data



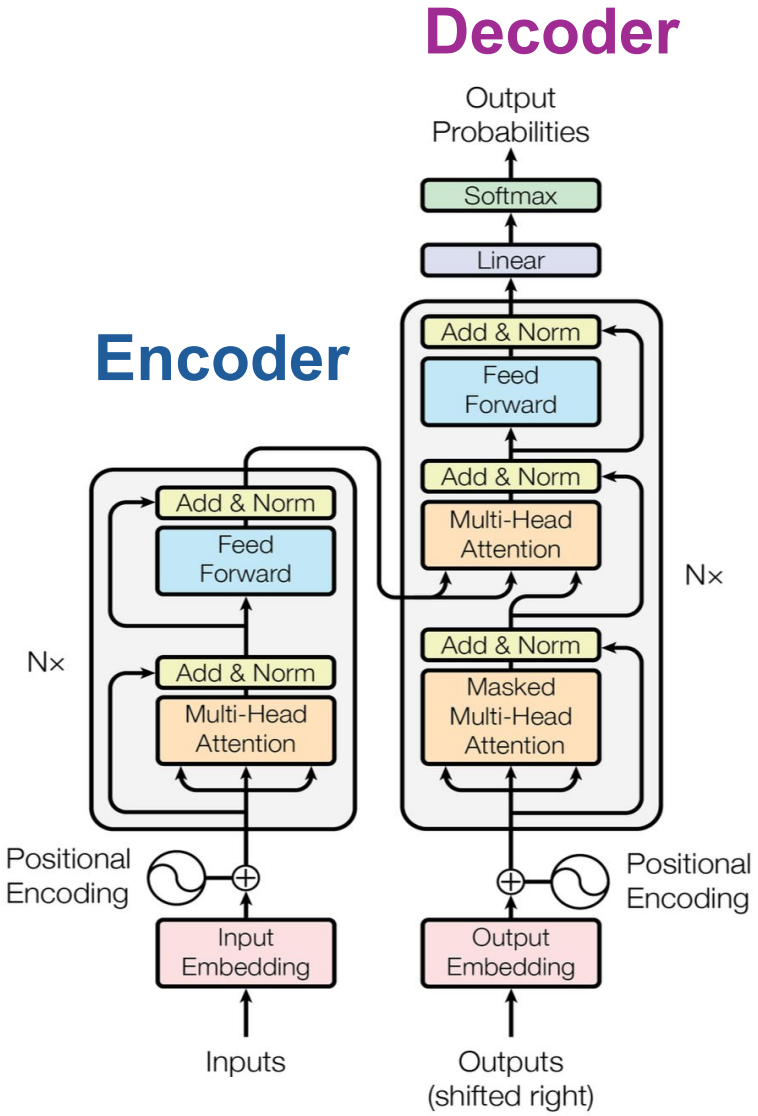


Figure 1: The Transformer - model architecture.

- Input Embedding: convert token embeddings
  - “king” [0.1, 0.2, ….]
- Positional Encoding: adding sequence order to embeddings
- Self-Attention: contextualized representation
-
- Multi-Head Attention: diverse context capture in multiple perspectives
- Feed Forward: nonlinear transformation and depth

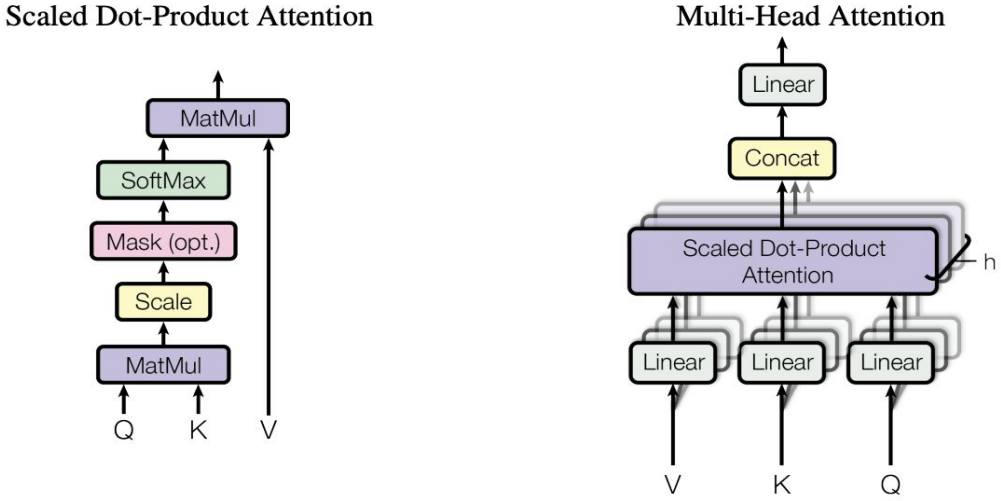
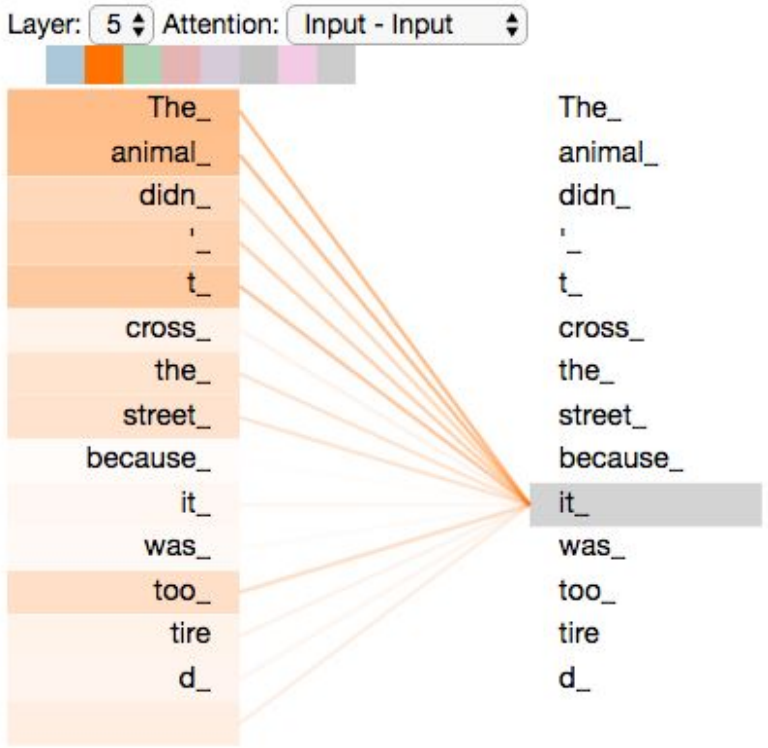


Figure 2: (left) Scaled Dot-Product Attention. (right) Multi-Head Attention consists of several attention layers running in parallel.

$$\text{Attention}(Q, K, V) = \text{softmax}(\frac{QK^T}{\sqrt{d_k}})V$$



- Attention: Focus on the most relevant tokens in the sequence
- Multi-Head Attention: Captures different types of relationships in parallel: **(“animal”** ← “it”**)** Resolves **coreference; (“tired”** ← “it”**)** Focuses on **reason/state; (“because”** ← “it”**)** Attends to **causal structure**

Transformer for Language Translation

- **Training** (Teacher Forcing): Decoder uses **real tokens** from the target sentence to learn next-token prediction.
  e.g., uses “你” to predict “好”
- **Loss function**: Cross-Entropy
- **Inference** (Autoregressive): Decoder uses its **own output** to predict the next token step by step.
  e.g., generates “你”, then uses it to predict “好”, then “吗”

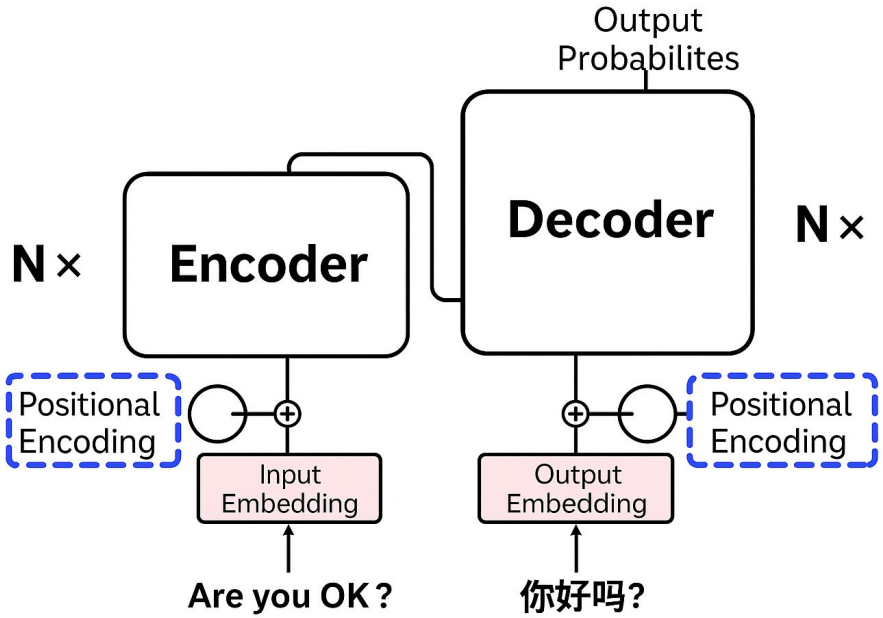
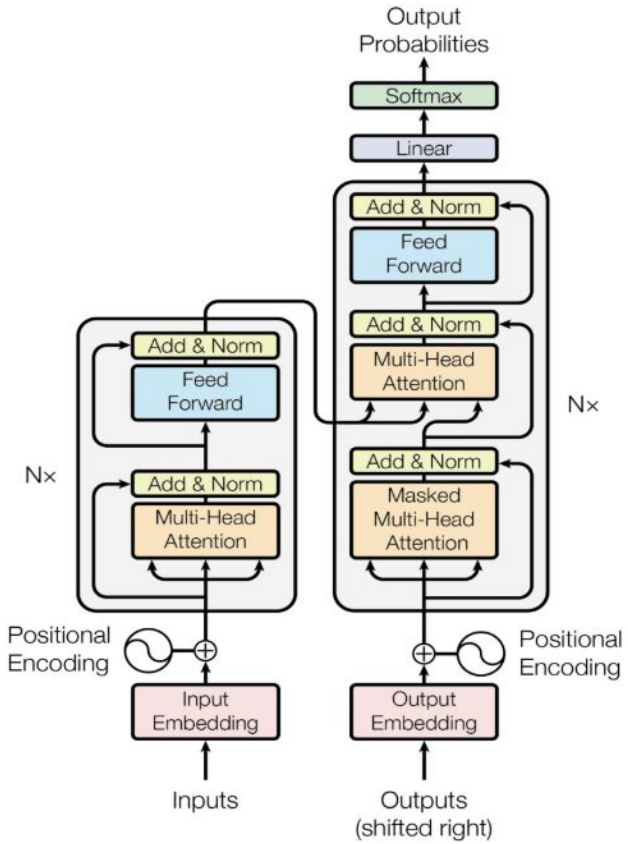


Table 2: The Transformer achieves better BLEU scores than previous state-of-the-art models on the English-to-German and English-to-French newstest2014 tests at a fraction of the training cost.

Model	BLEU		Training Cost (FLOPs)	
	EN-DE	EN-FR	EN-DE	EN-FR
ByteNet [15]	23.75			
Deep-Att + PosUnk [32]		39.2		$1.0 \cdot 10^{20}$
GNMT + RL [31]	24.6	39.92	$2.3 \cdot 10^{19}$	$1.4 \cdot 10^{20}$
ConvS2S [8]	25.16	40.46	$9.6 \cdot 10^{18}$	$1.5 \cdot 10^{20}$
MoE [26]	26.03	40.56	$2.0 \cdot 10^{19}$	$1.2 \cdot 10^{20}$
Deep-Att + PosUnk Ensemble [32]		40.4		$8.0 \cdot 10^{20}$
GNMT + RL Ensemble [31]	26.30	41.16	$1.8 \cdot 10^{20}$	$1.1 \cdot 10^{21}$
ConvS2S Ensemble [8]	26.36	**41.29**	$7.7 \cdot 10^{19}$	$1.2 \cdot 10^{21}$
Transformer (base model)	27.3	38.1	**$3.3 \cdot 10^{18}$**	
Transformer (big)	**28.4**	**41.0**	$2.3 \cdot 10^{19}$	

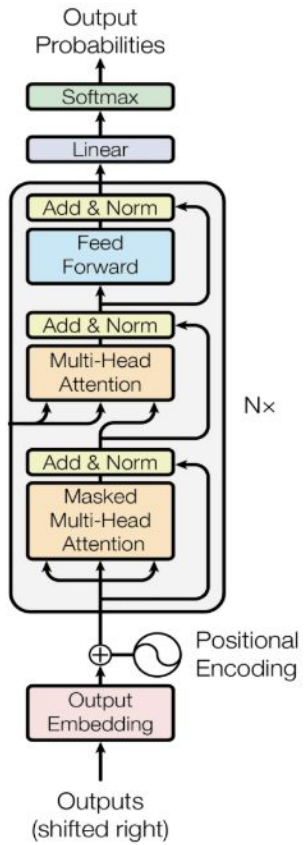
Model	Encoder/Decoder Layers	Embedding Size	Attention Heads	Total Parameters
Base	6	512	8	~65M
Big	6	1024	16	~213M

Transformer



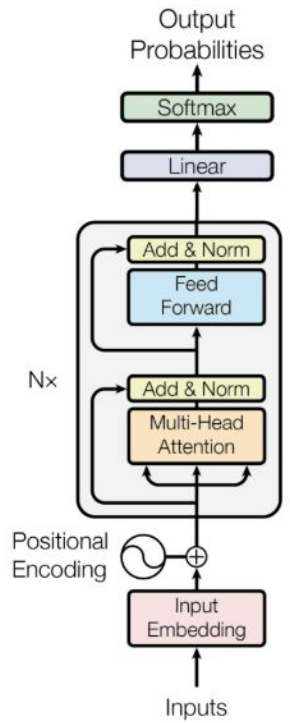
Encoder **Decoder**

GPT*



Decoder-only

BERT*



Encoder-only

*Illustrative example, exact model architecture may vary slightly

How BERT Works?

Encoder-only, trained with Marked Language Model (MLM)

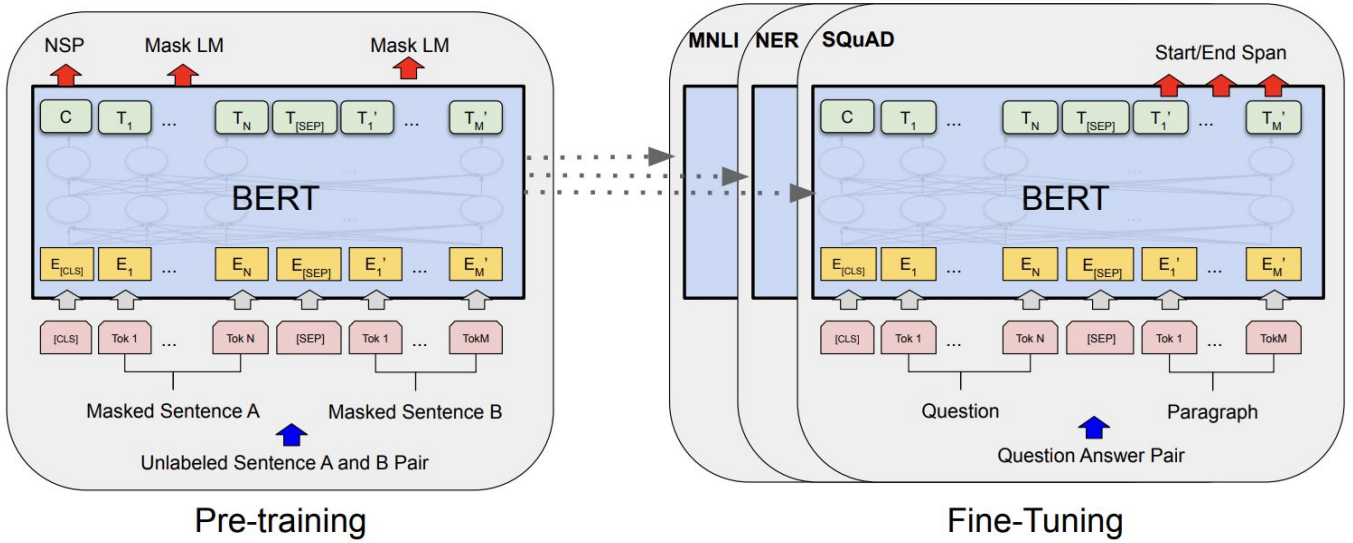


Figure 1: Overall pre-training and fine-tuning procedures for BERT. Apart from output layers, the same architectures are used in both pre-training and fine-tuning. The same pre-trained model parameters are used to initialize models for different down-stream tasks. During fine-tuning, all parameters are fine-tuned. `[CLS]` is a special symbol added in front of every input example, and `[SEP]` is a special separator token (e.g. separating questions/answers).

Encoder → Pre-training (Masked Language Model) → Finetuning on tasks

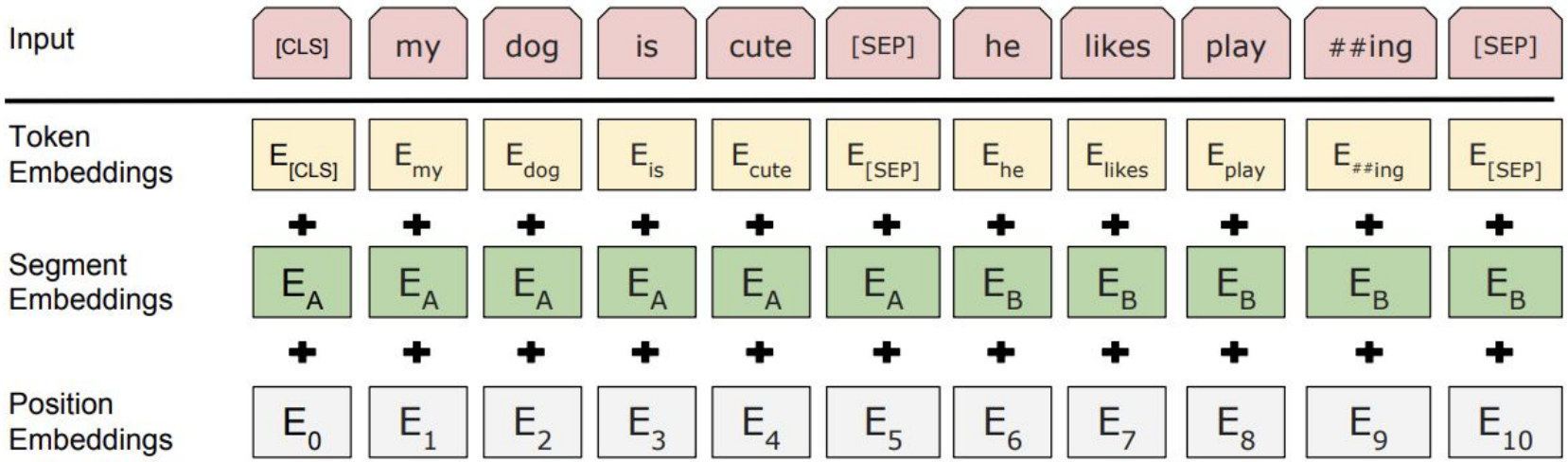


Figure 2: BERT input representation. The input embeddings are the sum of the token embeddings, the segmentation embeddings and the position embeddings.

How GPT Works?

Decoder-only, trained autoregressively

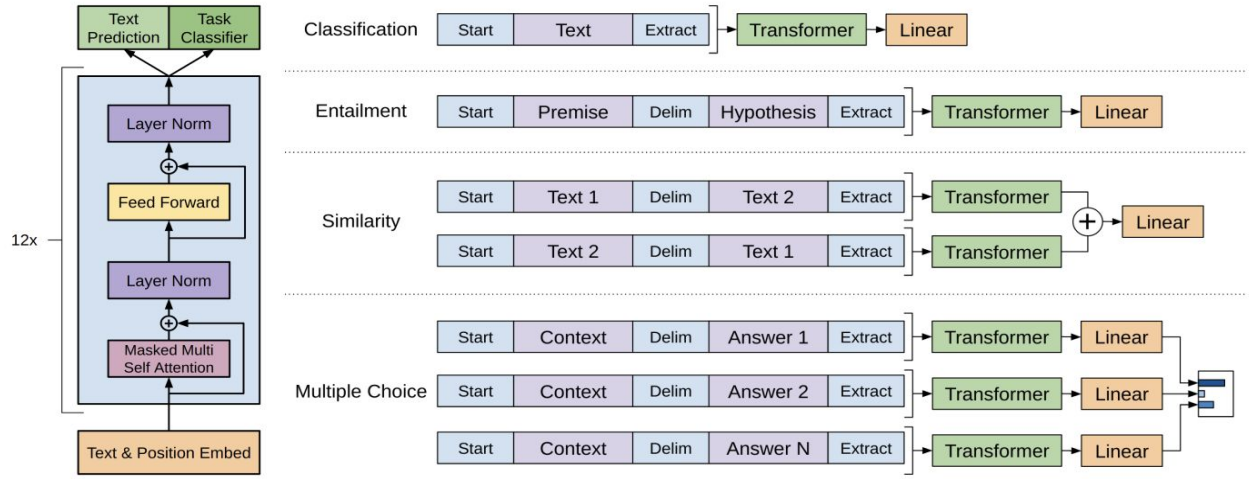
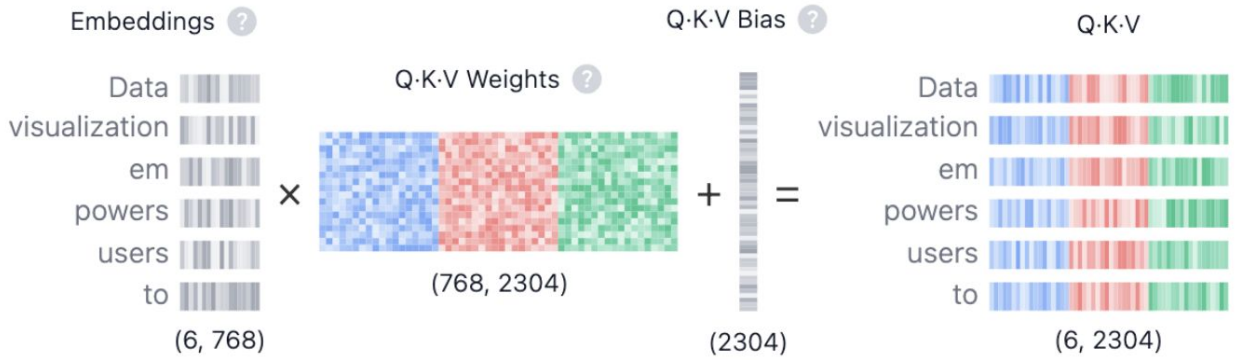


Figure 1: **(left)** Transformer architecture and training objectives used in this work. **(right)** Input transformations for fine-tuning on different tasks. We convert all structured inputs into token sequences to be processed by our pre-trained model, followed by a linear+softmax layer.

Decoder → Pre-training (Generative Model) → Finetuning on tasks

Deep Dive on Embeddings & Attention

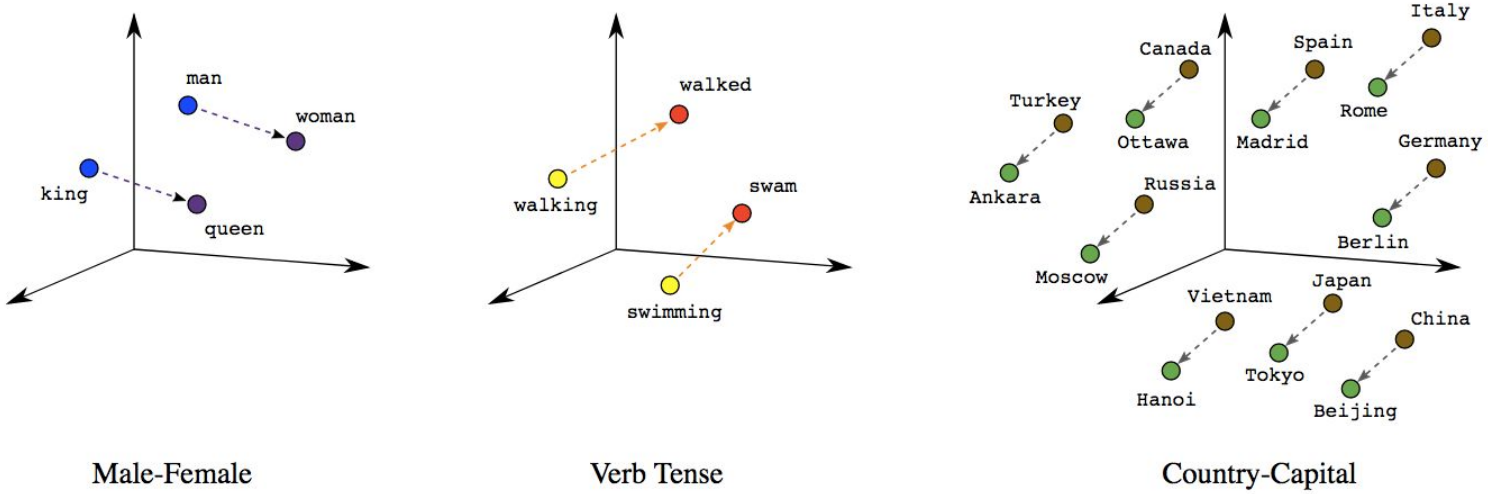
- Token and embeddings
- Key, Query, Value roles in attention



$$QKV_{ij} = (\sum_{d=1}^{768} \text{Embedding}_{i,d} \cdot \text{Weights}_{d,j}) + \text{Bias}_j$$

Word Embeddings

Definition: representing words as numerical vectors that captures their semantic and syntactic relationships

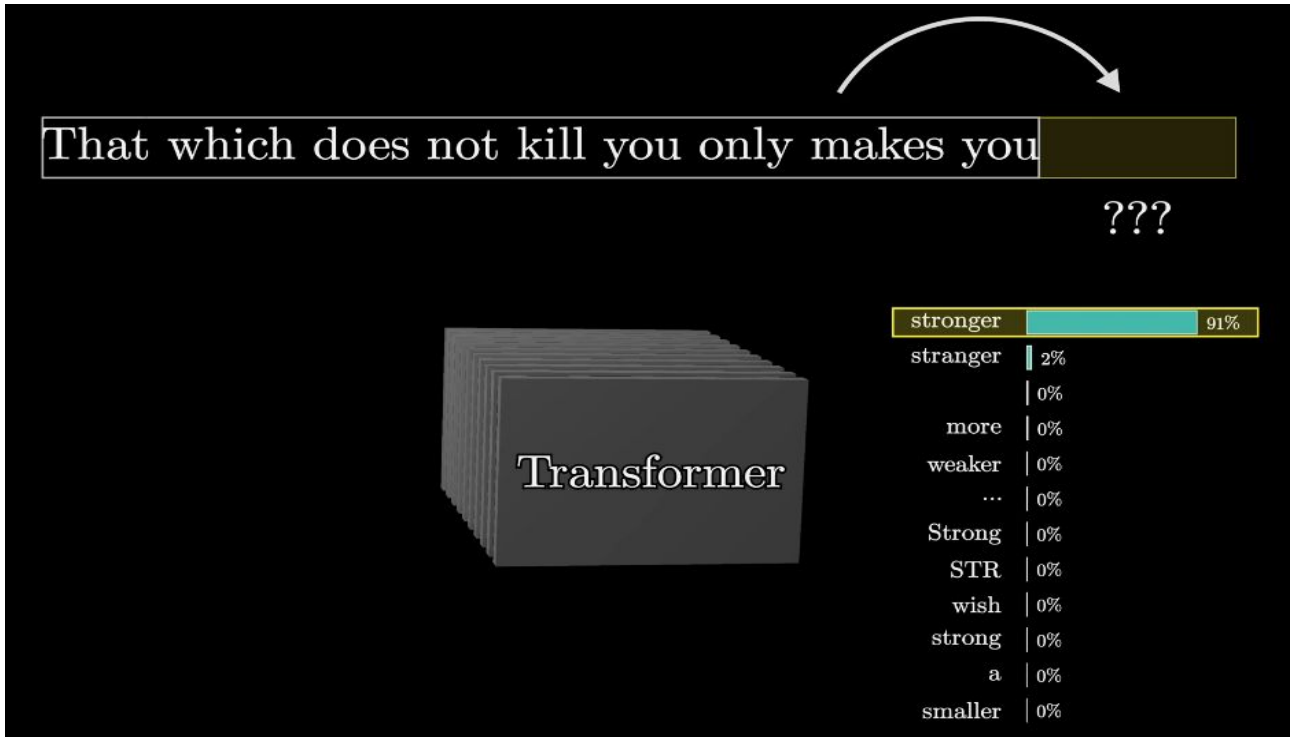


Male-Female Verb Tense Country-Capital

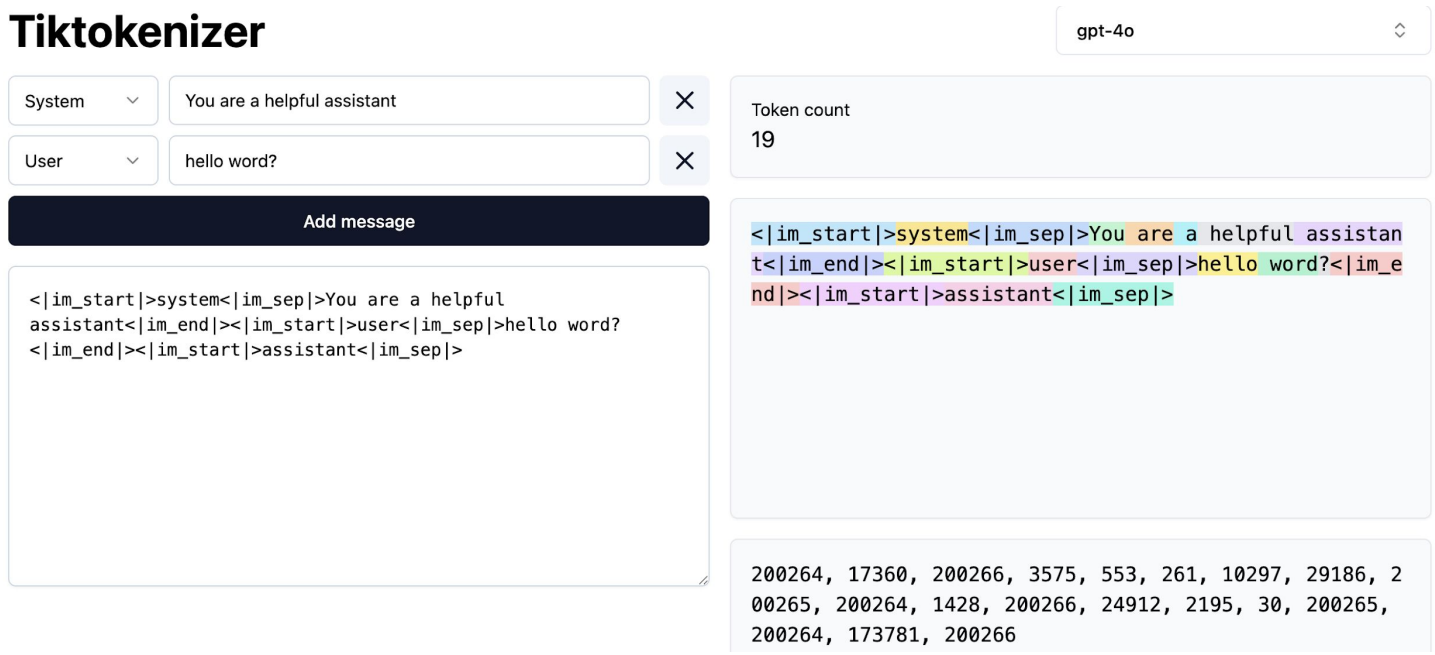
e.g. king = [0.7, 0.2, 0.9, …]; queen = [0.3, 0.7, 0.8, …]

Next Token Prediction

: core process for generation capabilities



Input Embeddings



Token	ID	Embedding (d_model = 4, simplified)
Hello	103	[0.25, -0.10, 0.45, 0.30]
world	205	[0.50, 0.60, -0.15, 0.10]

Positional Encoding

- It's a **vector added to each token embedding** to inject information about **its position** in the sequence.
- Types of Positional Encoding:
  - Fixed (Sinusoidal) — used in the original Transformer
  - Learned — used in BERT, GPT, etc.

$$PE_{(pos,2i)} = \sin(pos/10000^{2i/d_{model}})$$

$$PE_{(pos,2i+1)} = \cos(pos/10000^{2i/d_{model}})$$

"The cat sat"

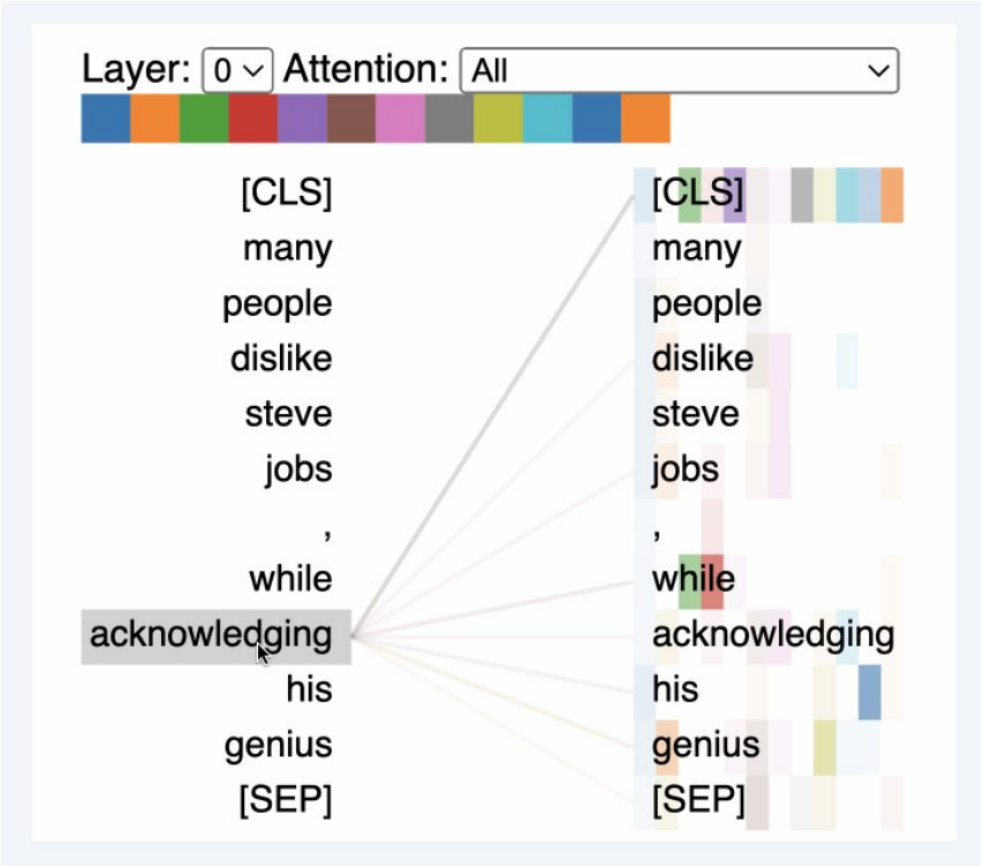
Position	Encoding
0	[0.0, 1.0, 0.0, 1.0]
1	[0.5, 0.0, 0.5, 0.0]
2	[1.0, 0.5, 1.0, 0.5]

Token	Embedding
The	[0.2, 0.5, 0.1, -0.3]
cat	[0.4, 0.2, 0.6, 0.0]
sat	[0.3, 0.6, -0.1, 0.2]

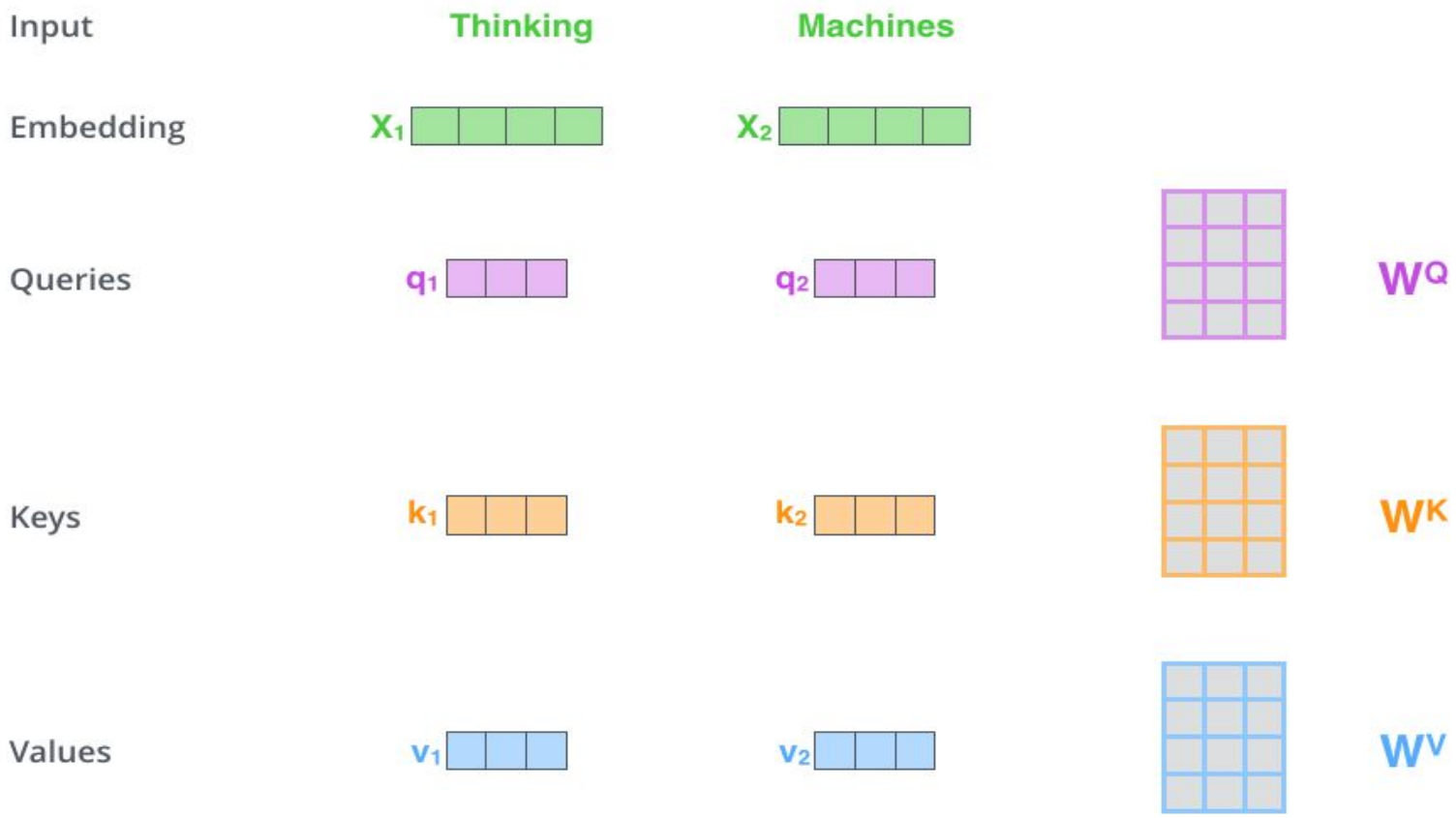
Final input = Embedding + Positional Encoding

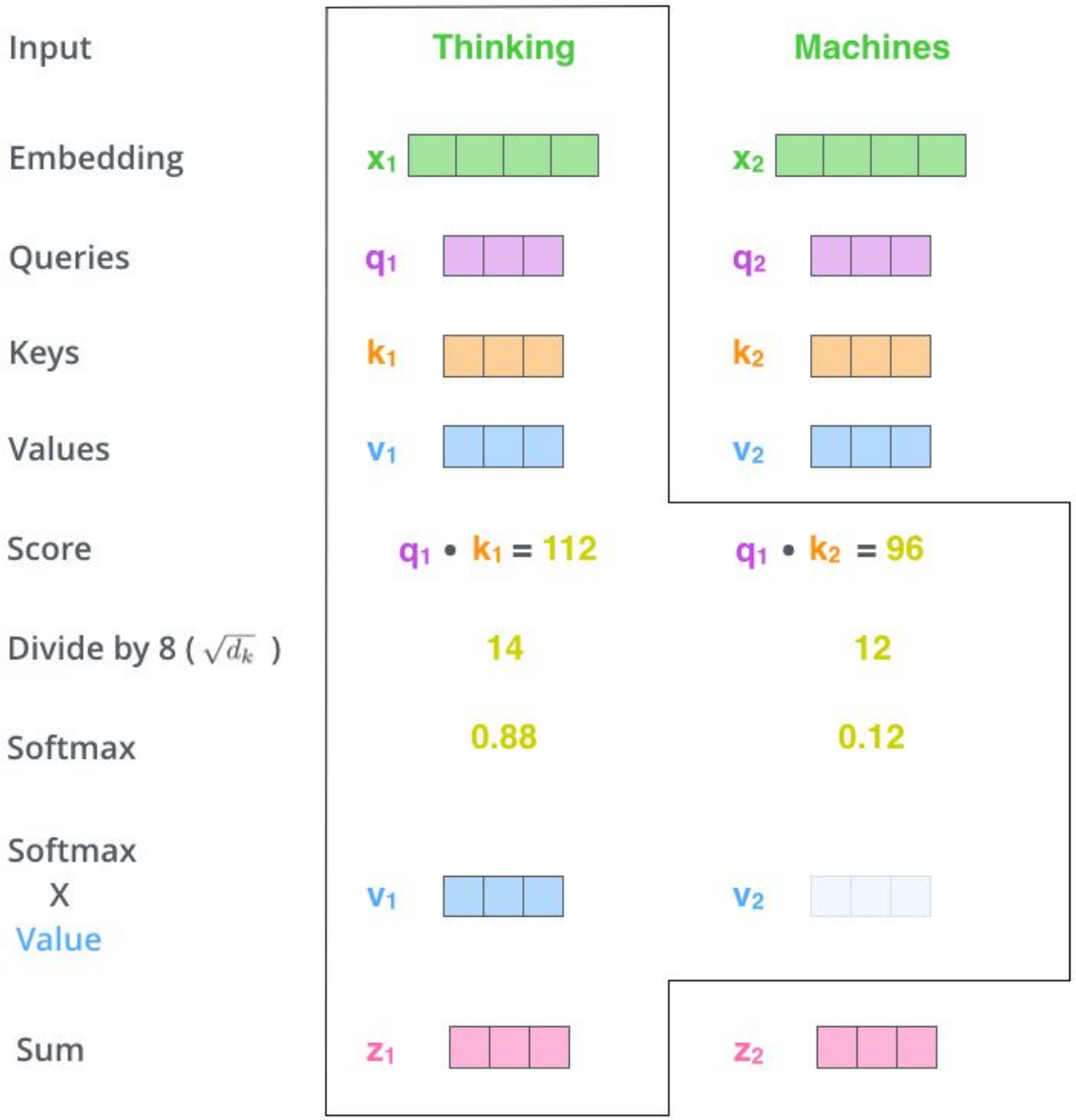
Attention in Transformers

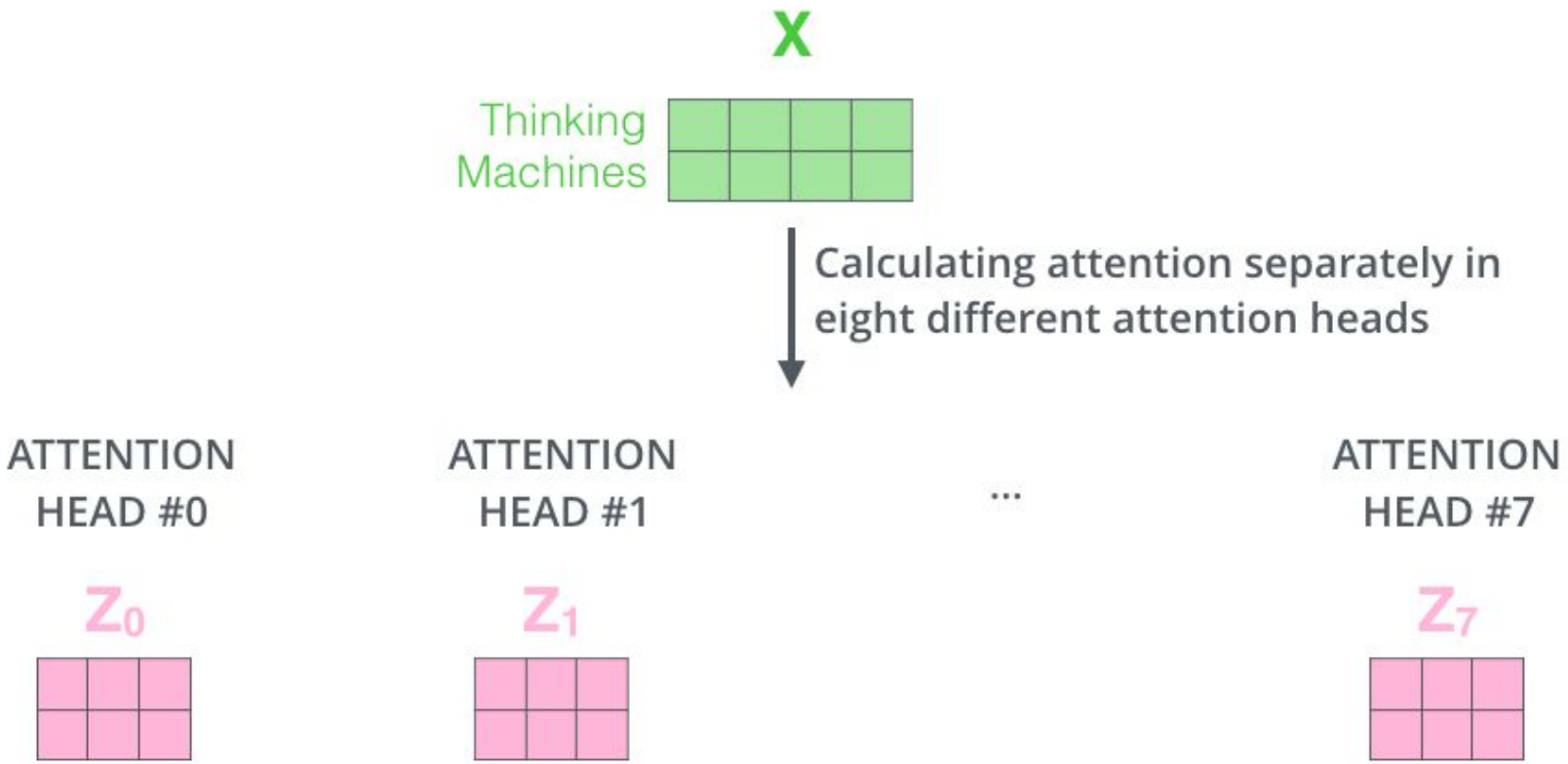
Self-attention: learns what to focus on for each token (words broken up)



How Attention Works




Input
Thinking
Machines
Embedding
x1
x2
Queries
q1
q2
Keys
k1
k2
Values
v1
v2
Score
q1 • k1 = 112
q1 • k2 = 96
Divide by 8 (√dk)
14
12
Softmax
0.88
0.12
Softmax X Value
v1
v2
Sum
z1
z2


X
Thinking
Machines
Calculating attention separately in
eight different attention heads
ATTENTION
HEAD #0
ATTENTION
HEAD #1
...
ATTENTION
HEAD #7
Z0
Z1
Z7

1) This is our input sentence*

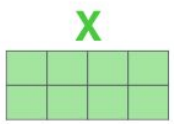
2) We embed each word*

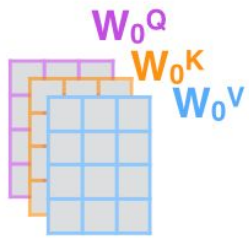
3) Split into 8 heads. We multiply X or R with weight matrices

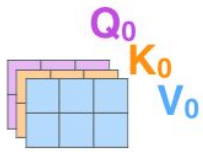
4) Calculate attention using the resulting Q/K/V matrices

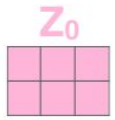
5) Concatenate the resulting Z matrices, then multiply with weight matrix W^O to produce the output of the layer

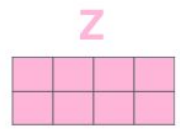
Thinking Machines



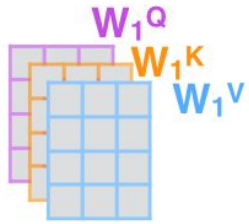


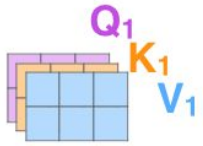


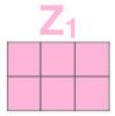




* In all encoders other than #0, we don't need embedding. We start directly with the output of the encoder right below this one



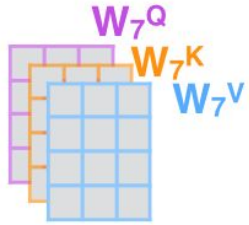


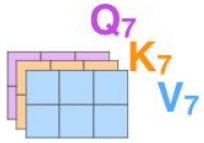


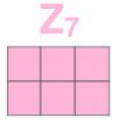
...

...

...



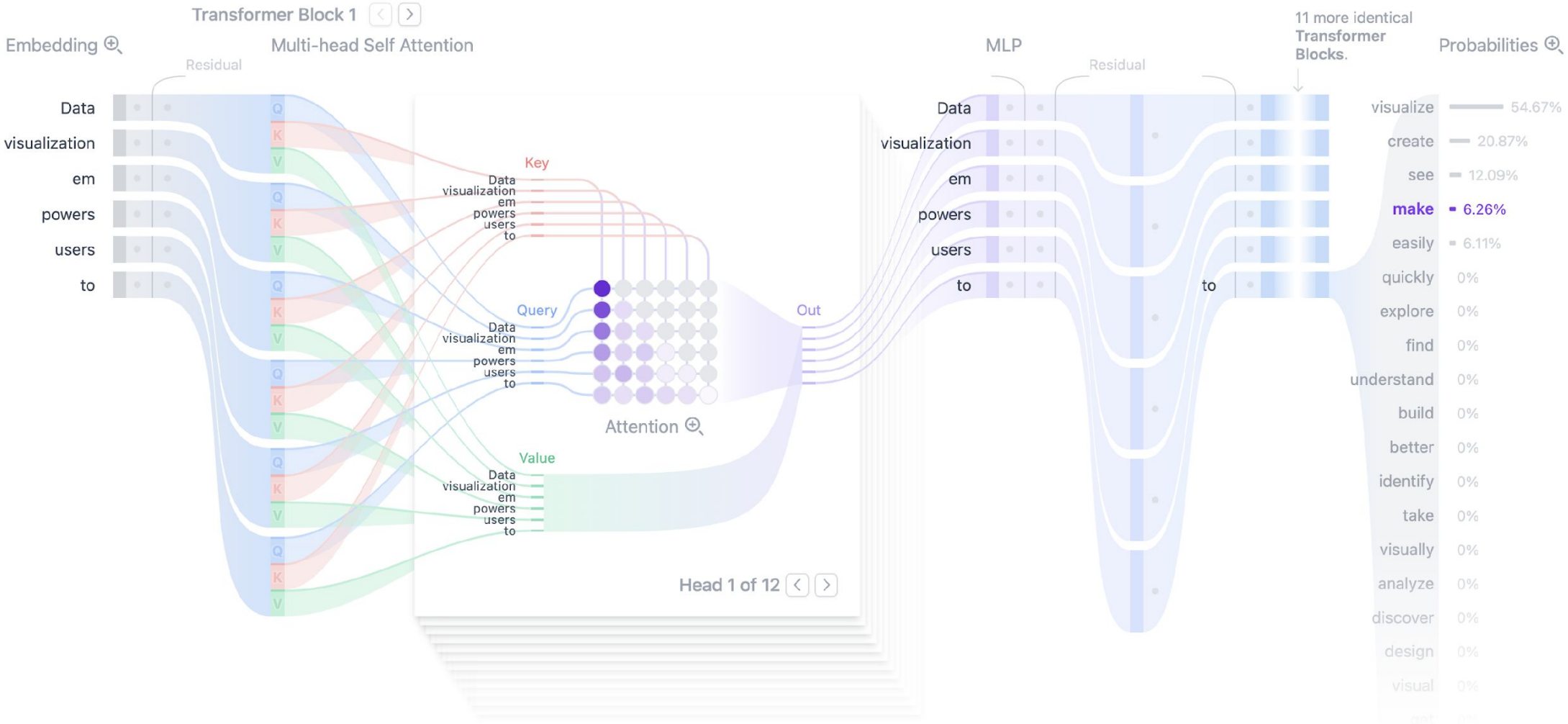




Analogy for Attention Understanding

- Problem: “What should this token pay attention to — and how much?”
- Three vectors:
  - Query (Q) What I’m **looking for**
  - Key (K) What I **offer**
  - Value (V) What I **carry as information**
- Analogy: Finding a Book in a Library
  - **Query (Q)**: Your search query (“topic = attention mechanisms”)
  - **Key (K)**: Each book’s title & summary
  - **Value (V)**: The actual content of the book
  - Result: compare your **Q** to every **K**, and select the **most relevant V**.

Transformer Visualization



Count Parameters for GPT Model

Total weights: 175,181,291,520
Organized into 27,938 matrices

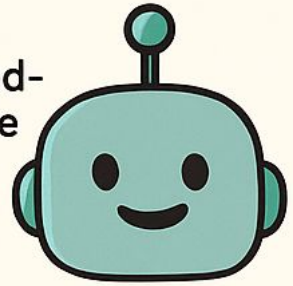
GPT-3

Embedding	$d\_embed^{12,288} * n\_vocab^{50,257}$	$= 617,558,016$
Key	$d\_query^{128} * d\_embed^{12,288} * n\_heads^{96} * n\_layers^{96}$	$= 14,495,514,624$
Query	$d\_query^{128} * d\_embed^{12,288} * n\_heads^{96} * n\_layers^{96}$	$= 14,495,514,624$
Value	$d\_value^{128} * d\_embed^{12,288} * n\_heads^{96} * n\_layers^{96}$	$= 14,495,514,624$
Output	$d\_embed^{12,288} * d\_value^{128} * n\_heads^{96} * n\_layers^{96}$	$= 14,495,514,624$
Up-projection	$n\_neurons^{49,152} * d\_embed^{12,288} * n\_layers^{96}$	$= 57,982,058,496$
Down-projection	$d\_embed^{12,288} * n\_neurons^{49,152} * n\_layers^{96}$	$= 57,982,058,496$
Unembedding	$n\_vocab^{50,257} * d\_embed^{12,288}$	$= 617,558,016$

Transformer Architecture and GenAI Models

- The Transformer is the foundational neural network architecture introduced in “Attention Is All You Need” (2017).
- Generative AI (GenAI) models like GPT-4, Claude, Gemini, LLaMA are all built on the Transformer architecture, fine-tuned for specific tasks.

GPT-4



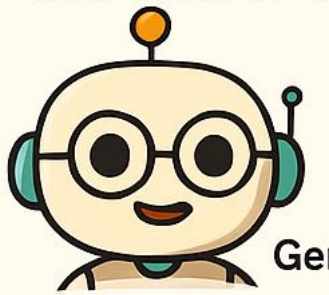
- Closed-source
- API access

- Strong general capabilities
- GPT-4o adds audio, vision, real-time
- 128k contentt window
- Reinforcement Learnirg from Human Feedback (RLHF)

Claude 3

- Constitutional AI principles
- Focused on safety anlignmennt
- 200k context window
- Strong at reasoning & summarization

Gemini 1.5



Gemini 1.5

- Multimodal (text, vision, audio)
- 1M context window (longest)
- Encoder-decoder architecture + MoE
- Integrated into Google products

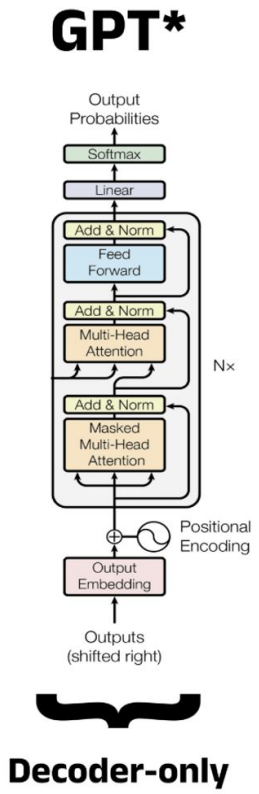
LLaMA 3

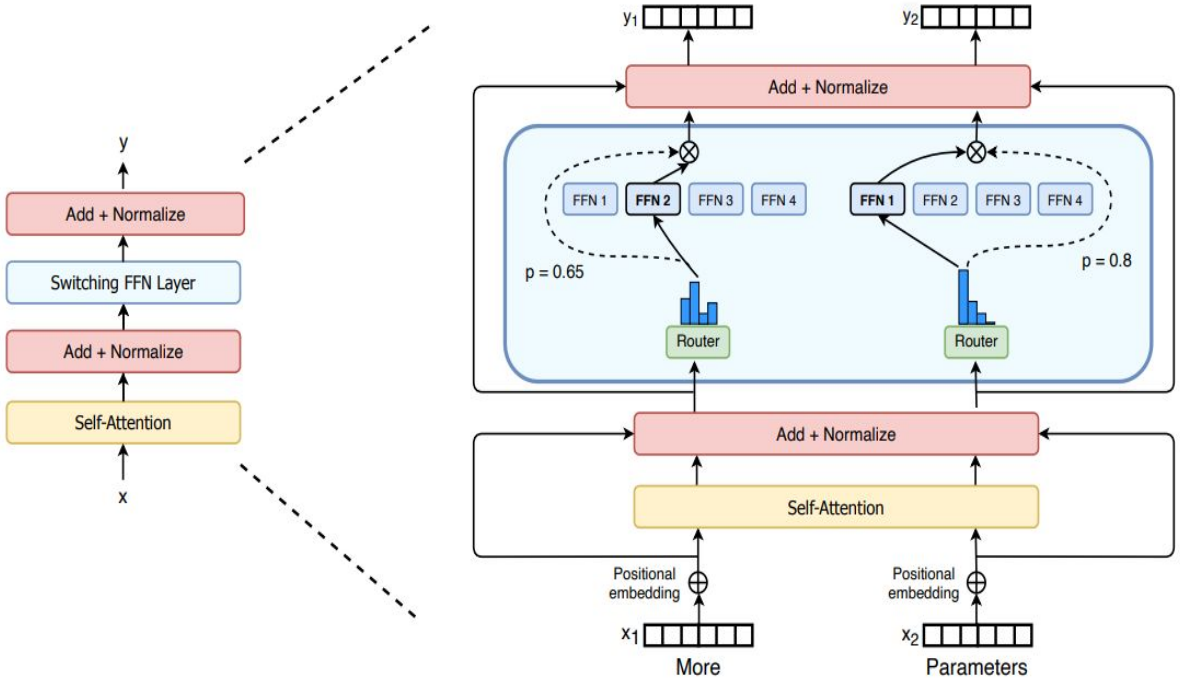
- Open-source model family

- Community fine-tuning
- Text-only model
- Lightweight & scalable
- Ideal for academic OSS use

Mixture of Experts (MoE) Architecture

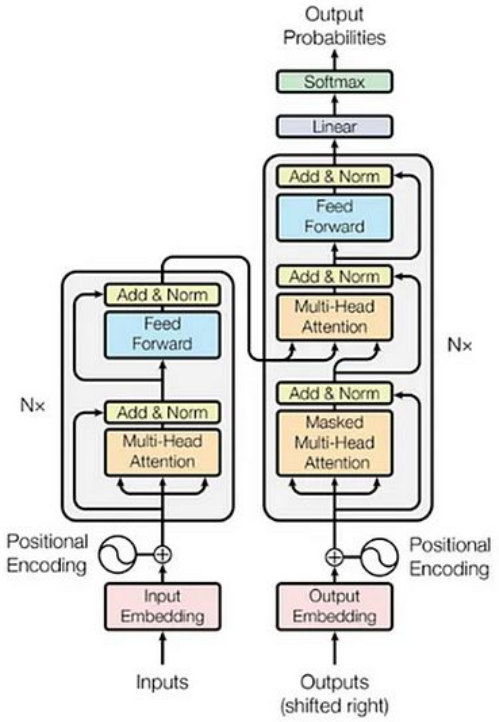
sparse model architecture where, instead of activating all parts of a neural network for every input, only a small subset of “experts” (specialized sub-networks) are used per input.



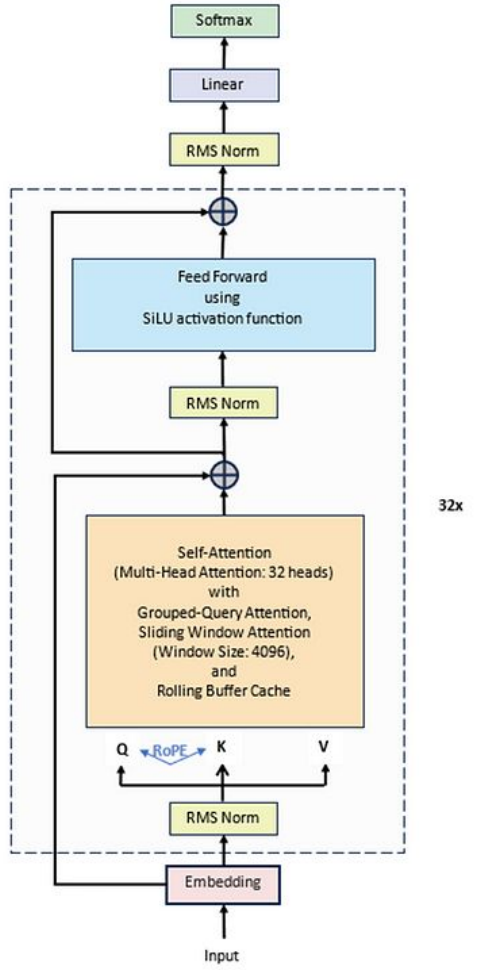


What Gains at MoE

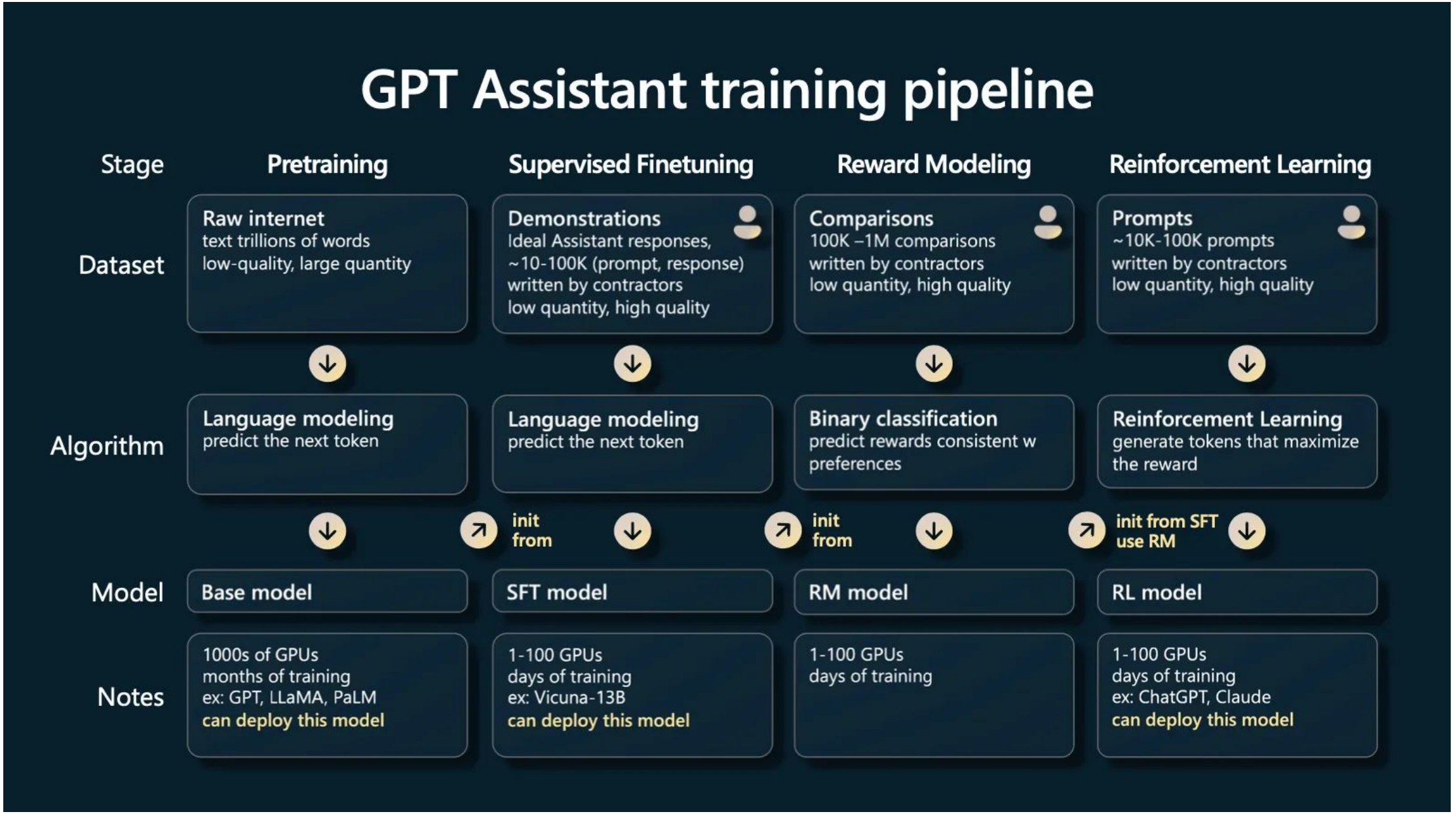
- 1. **Computational Efficiency**
- 2. **Scalable Parameter Growth**
- 3. **Specialization by Experts**
- 4. **Multimodal Benefits**



Transformer



Mistral 7B


GPT Assistant training pipeline
Stage
Pretraining
Supervised Finetuning
Reward Modeling
Reinforcement Learning
Dataset
Raw internet
text trillions of words
low-quality, large quantity
Demonstrations
Ideal Assistant responses,
~10-100K (prompt, response)
written by contractors
low quantity, high quality
Comparisons
100K –1M comparisons
written by contractors
low quantity, high quality
Prompts
~10K-100K prompts
written by contractors
low quantity, high quality
Algorithm
Language modeling
predict the next token
Language modeling
predict the next token
Binary classification
predict rewards consistent w
preferences
Reinforcement Learning
generate tokens that maximize
the reward
init
from
init
from
init from SFT
use RM
Model
Base model
SFT model
RM model
RL model
Notes
1000s of GPUs
months of training
ex: GPT, LLaMA, PaLM
can deploy this model
1-100 GPUs
days of training
ex: Vicuna-13B
can deploy this model
1-100 GPUs
days of training
1-100 GPUs
days of training
ex: ChatGPT, Claude
can deploy this model

Pre-training	**Supervised Finetuning**	**RLHF**
Input: "The capital of France is..."	**Prompt**: "What's the weather like in Paris?"	**Prompt**: "Explain quantum physics like I'm five."
Output: "Paris"	**Response**: "I can't access live data, but... "	"**Quantum physics** is like magic rules for tiny things — like atoms and particles. …"

🔁 Stage-by-Stage Comparison with Examples

Stage	Dataset Type	Example Input	Model Behavior	Goal
1. Pretraining	Trillions of tokens from internet (low-quality, high-volume)	"The capital of France is..."	Predicts: **"Paris"**	Learn general language and knowledge
2. Supervised Finetuning (SFT)	Handwritten Q&A pairs (10k–100k, high-quality)	**Prompt**: "What's the weather like in Paris?" **Response**: "I can't access live data, but..."	Learns how to behave like an assistant	Make model helpful, safe, honest
3. Reward Modeling (RM)	Pairwise comparisons of responses	Response A: "Paris is nice." Response B: "Paris is the capital of France." → **Choose B**	Learns to score outputs based on human preference	Train a **reward model**
4. Reinforcement Learning (RLHF)	Prompts + RM scores	Prompt: "Explain quantum physics like I'm five." Model generates multiple outputs → RM ↓ s best → **RL maximizes reward**	Model improves generation using RL	Align model behavior with humans

Take-home Messages

- The Transformer architecture consists of an **encoder** and a **decoder**, built around core components like **token embeddings**, **positional embeddings**, **attention mechanisms**, and **multi-head attention**.
- The **Key, Query, and Value** mechanism enables each token to determine **which other tokens are relevant**, allowing the model to build contextual understanding.
- Modern generative AI models—such as GPT—are primarily based on the **decoder** structure of Transformers, optimized for text generation. To make these models perform well in **dialogue and conversational tasks**, they are further refined using **supervised fine-tuning** and **reinforcement learning from human feedback (RLHF)**.

References

- [1706.03762] Attention Is All You Need
- How Transformers Work: A Detailed Exploration of Transformer Architecture
- Large language models, explained with a minimum of math and jargon
- transformer-explainer
- Transformers, the tech behind LLMs
- Andrej Karpathy: State of GPT | BRK216HFS
- Andrej Karpathy: Let's build GPT: from scratch, in code, spelled out

Q & A